Kristianstad University

Embedded Systems

LAB 4 Report

Interrupts, Timers, UART, and Embedded Control

Name: Gagan Adhikari
Course: Embedded Systems
Date: June 7, 2026

1 Authorship Statement

Authorship Statement I Gagan Adhikari declare that this lab report is a result of my own original. Explanation of Work: All of the task for the lab was done by me and All of this reports is also done by me except for some syntax that I asked AI to create for the Latex. For the Labs I did use AI just to get verified my work atleast to be on a safe side before Labs and helped with flowcharts(Eraser AI and Draw io).

Contents

1 Authorship Statement **1**

2 Task 1 – Blinking LED using Timer Interrupt **5**

2.1 Task Summary 5

2.2 Timer Configuration and Calculations 5

2.3 Answers to Task Questions 5

2.4 Explanation of Operation 5

2.5 Circuit Design 7

2.6 Code Reference 7

3 Task 2 – UART-Controlled LED Blinking **8**

3.1 Task Summary 8

3.2 USART Configuration 8

3.3 Interrupt Behaviour 8

3.4 Explanation of Operation 8

3.5 Code Reference 9

4 Task 3 – Button Press Using Polling **10**

4.1 Task Summary 10

4.2 Design Choices 10

4.3 Explanation of Operation 11

4.4 Circuit Design 12

4.5 Code Reference 12

5 Task 4 – Button Press Using External Interrupt **13**

5.1 Task Summary 13

5.2 External Interrupt Configuration 13

5.3 Answers to Task Questions 13
5.4 Explanation of Operation 13
5.5 Circuit Design And Code Explanation 15
5.6 Code Reference 15

6 Task 5 – Traffic Light Using Software Delays 15

6.1 Task Summary 15
6.2 Handling Long Delays 15
6.3 Explanation of Operation 17
6.4 Traffic Light Circuit and Code 18
6.5 Code Reference 18

7 Task 6 – Cycling LEDs Using Button + Timer-Based Debouncing 19

7.1 Task Summary 19
7.2 Design and Optimization 19
7.3 Explanation of Operation 19
7.4 Code Reference 20

8 Task 7 – Interrupt-Based Communication Between Two Arduinos 20

8.1 Introduction 20
8.2 System Architecture 20
8.3 Safety Considerations 21
8.4 Code Reference 21

Appendix 21

A Task 1 Code 21

B Task 2 Code 22

C Task 3 Code 24

D Task 4 Code **25**

E Task 5 Code **25**

F Task 6 Code **26**

G Task 7 Code **28**

2 Task 1 – Blinking LED using Timer Interrupt

2.1 Task Summary

The goal of Task 1 is to configure Timer1 on the ATmega328P to generate a Compare Match A interrupt every 1 s. Inside the interrupt service routine (ISR), the LED on PB5 toggles, producing a blink frequency of 0.5 Hz. The main loop remains empty, demonstrating a fully interrupt-driven design.

2.2 Timer Configuration and Calculations

Timer1 is configured in CTC (Clear Timer on Compare Match) mode using the `WGM12` bit. With a 16 MHz CPU clock and a prescaler of 1024:

$$f_{\text{timer}} = \frac{16\,000\,000}{1024} = 15625 \text{ Hz}$$

To obtain a 1 s interval:

$$\text{OCR1A} = 15625 - 1 = 15624$$

Thus, the timer reaches the compare value once per second, triggering the ISR.

2.3 Answers to Task Questions

- **OCR1A value:** 15624.
- **Timer mode:** CTC mode (`WGM12 = 1`).
- **Prescaler:** 1024 (`CS12 = 1`, `CS10 = 1`).
- **Local interrupt activation:** `TIMSK1 |= (1 << OCIE1A)`.
- **Global interrupt activation:** `sei()`.

2.4 Explanation of Operation

`From the figure 1`, The program configures PB5 as an output and sets up Timer1 in CTC mode with OCR1A = 15624. A prescaler of 1024 is applied, and the Timer1 compare-match interrupt is enabled. Global interrupts are activated, and the main loop runs indefinitely without work. Each time the timer reaches the compare value, the ISR toggles PB5, producing a 1-second blink.

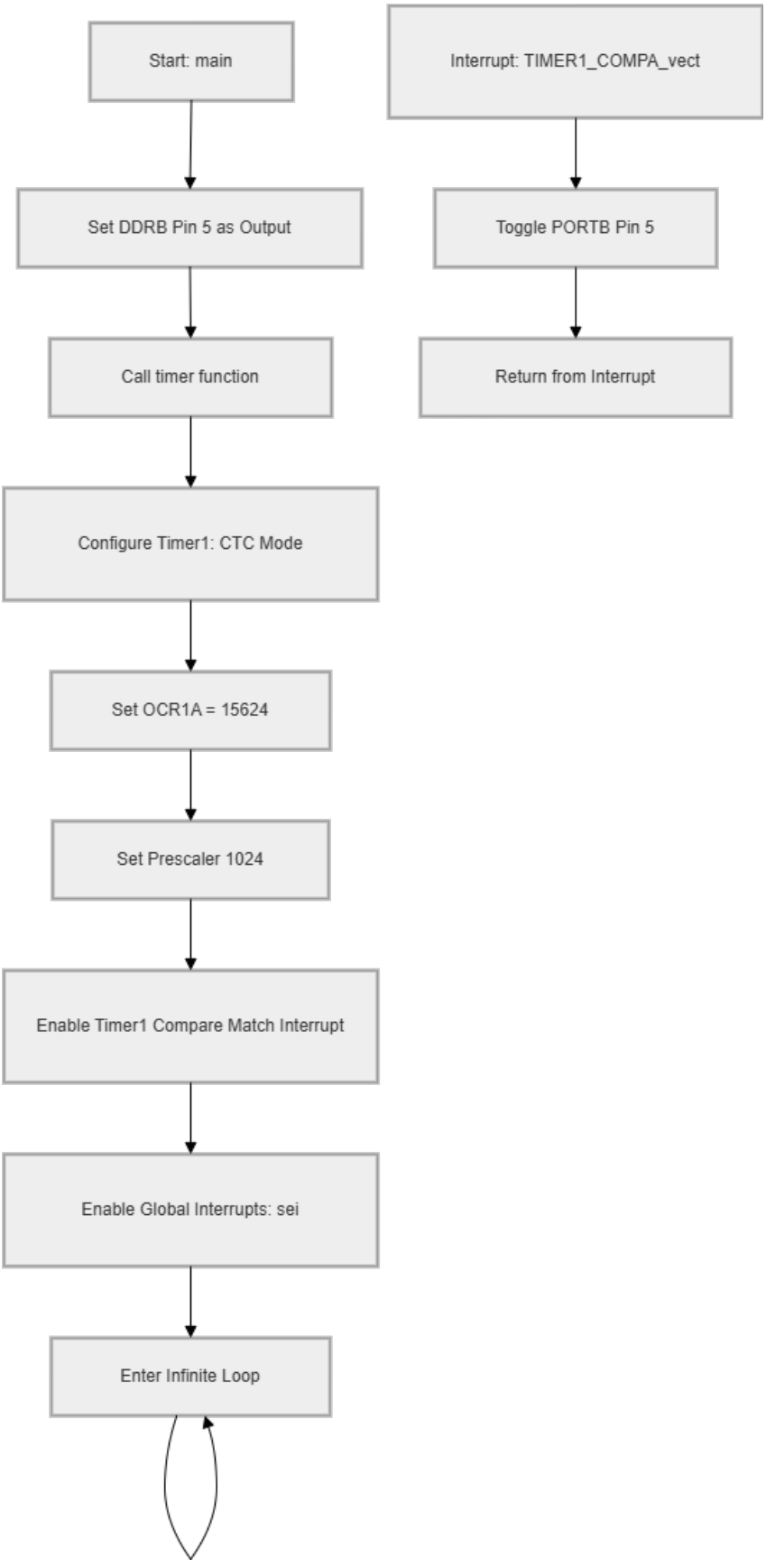


Figure 1: Flowchart for Task 1.

2.5 Circuit Design

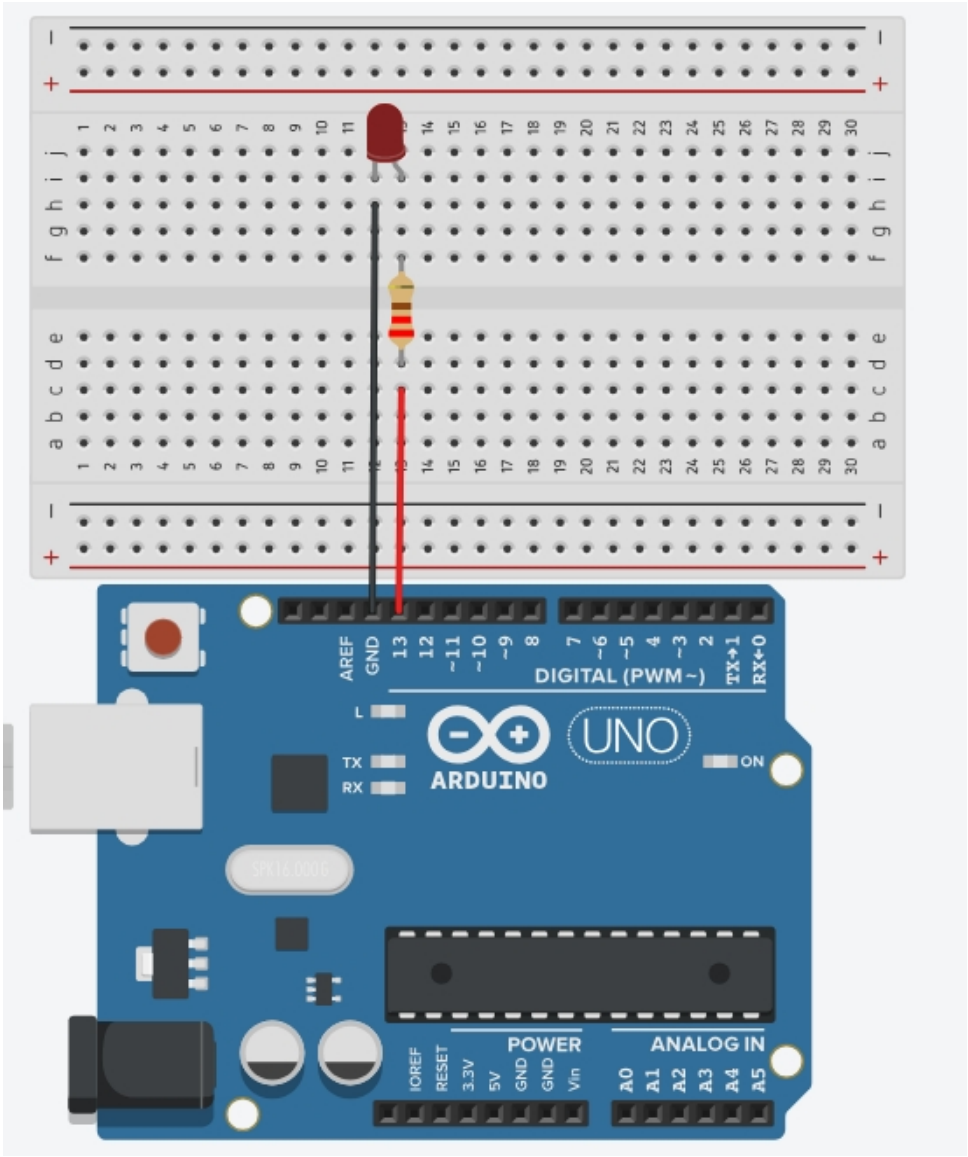


Figure 2: Circuit diagram for Task 1 and Task 2.

`In figure 2`, The Arduino pin drives an LED through a current-limiting resistor. Digital pin 13 connects to the LED's anode, and the LED's cathode goes to GND. The resistor protects the LED by limiting current. This setup allows the microcontroller to turn the LED ON and OFF using a digital output signal.

2.6 Code Reference

See Appendix A.

3 Task 2 – UART-Controlled LED Blinking

3.1 Task Summary

Task 2 is kind of similar to task 1 just with UART communication. The PC sends characters to the microcontroller:

- Sending `'1'` enables blinking.
- Sending `'0'` disables blinking and forces the LED off.

All logic is interrupt-driven: UART RX interrupt updates a flag, and Timer1 toggles the LED based on that flag.

3.2 USART Configuration

The USART is configured for:

- 9600 baud
- 8 data bits, 1 stop bit, no parity (8N1)
- Receiver enabled
- RX interrupt enabled

3.3 Interrupt Behaviour

The `USART_RX_vect` ISR reads the received byte:

- `'1'` → blinking enabled
- `'0'` → blinking disabled, LED forced LOW

Timer1 runs identically to Task 1.

3.4 Explanation of Operation

The Figure 3 shows flowchart, that the program begins in main() and performs all hardware setup during initialization. It configures PB5 as an output, initializes USART for serial communication, and sets up Timer1. Global interrupts are then enabled so both ISRs can run. After initialization, the program enters an infinite loop where no active work is done. All real functionality happens inside interrupts during runtime.

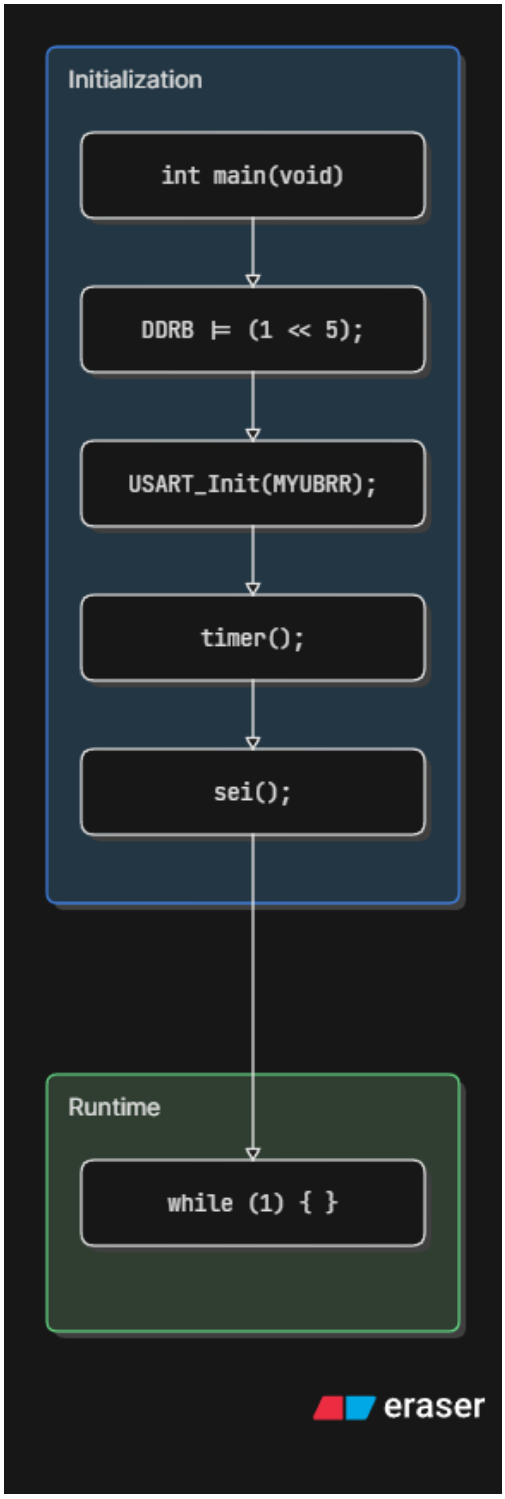


Figure 3: Flowchart for Task 2.

3.5 Code Reference

See Appendix B.

4 Task 3 – Button Press Using Polling

4.1 Task Summary

Task 3 implements button-controlled LED toggling using polling rather than interrupts. The button is connected to PD7 with an internal pull-up resistor. A software debounce is implemented using Timer1.

4.2 Design Choices

A pull-up configuration ensures:

- Not pressed → HIGH
- Pressed → LOW

A short 10 ms delay filters mechanical bouncing.

4.3 Explanation of Operation

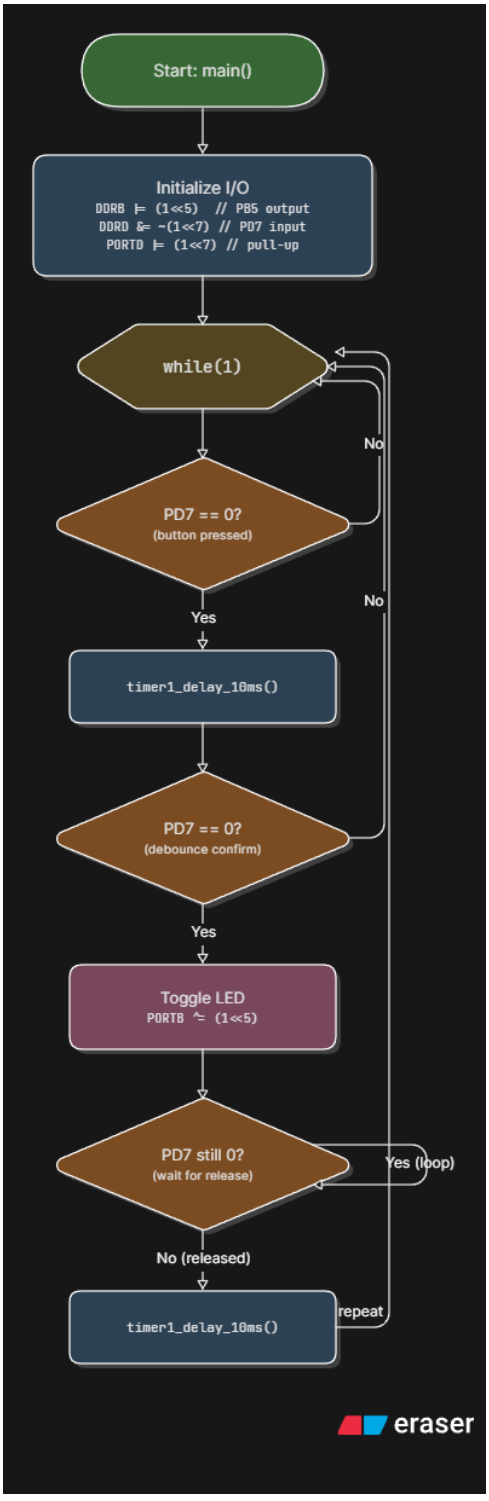


Figure 4: Flowchart for Task 3.

The figure 4 shows, that the program continuously checks the input pin `PD7` inside the main loop. When `PD7` reads LOW, the code calls `timer1_delay_10ms()` to debounce the button. If the pin is still LOW after debouncing, the LED on `PB5` is toggled using `PORTB ^= (1<<5)`. The flowchart then enters a loop that waits until `PD7` returns HIGH,

indicating button release. After a final 10 ms debounce, the program resumes polling and repeats the process.

4.4 Circuit Design

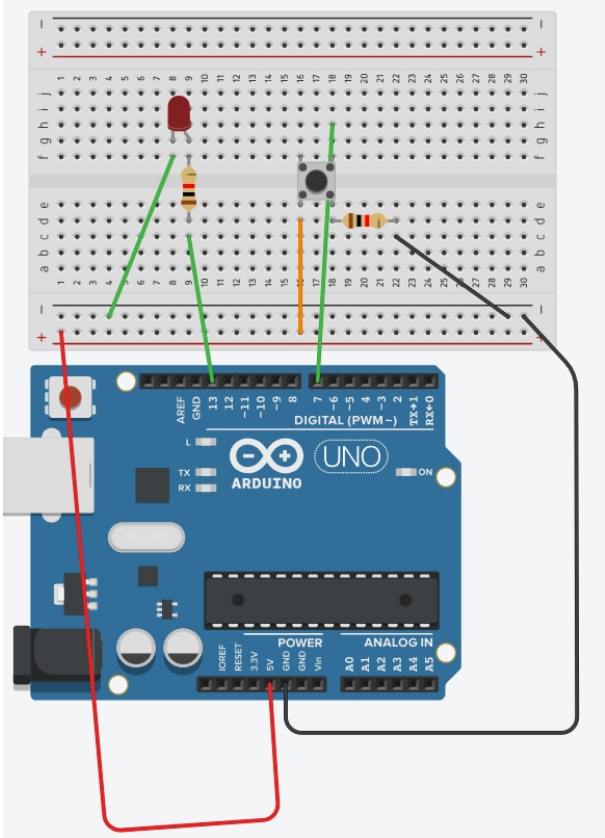


Figure 5: Circuit Design for task 3 and 4

In the Figure 5 shows,The circuit for this program uses one input pin and one output pin on the microcontroller. Pin `PB5` is configured as an output and drives an LED through a current–limiting resistor to ground. Pin `PD7` is configured as a digital input, and an internal pull–up resistor is enabled using `PORTD |= (1 << 7)`. This means the pin normally reads HIGH and goes LOW only when the push button is pressed and connected to ground. The program continuously monitors `PD7`; when a LOW level is detected, a 10 ms software debounce is applied using `timer1_delay_10ms()`. If the button is still pressed after debouncing, the LED on `PB5` is toggled. The code then waits until `PD7` returns HIGH (button released) before applying another debounce and resuming normal polling.

4.5 Code Reference

See Appendix C.

5 Task 4 – Button Press Using External Interrupt

5.1 Task Summary

Task 4 introduces the use of external interrupts on the ATmega328P. A push button is connected to PD2 (INT0), and an LED is connected to PD7. When the button is pressed, INT0 triggers an interrupt service routine (ISR) that toggles the LED. The main loop remains empty, demonstrating a fully interrupt-driven design.

5.2 External Interrupt Configuration

The ATmega328P provides two external interrupts: INT0 (PD2) and INT1 (PD3). This task uses INT0, configured to trigger on the falling edge of the button signal.

5.3 Answers to Task Questions

- **1. How is the local interrupt activated?** By enabling INT0 in the External Interrupt Mask Register:

  ```
  EIMSK |= (1 << INT0);
  ```

- **2. How is EICRA configured and why?** To trigger on a falling edge:

  ```
  ISC01 = 1, ISC00 = 0
  ```

  This ensures the interrupt fires when the button transitions from HIGH to LOW.

- **3. Which ISR is used?**

  ```
  ISR(INT0_vect)
  ```

5.4 Explanation of Operation

Figure 6 shows the program beginning in `main()`, where all hardware configuration is performed. First, `DDRD` is set so that `PD7` becomes an output and `PD2` becomes an input, matching the program's I/O setup. Next, the external interrupt is configured by selecting a falling-edge trigger in `EICRA`, followed by enabling interrupt `INT0` in `EIMSK`. Global interrupts are then activated so the external interrupt can fire during execution. Finally, the program enters an infinite `while(1)` loop, where it waits while all real actions occur inside the interrupt service routine.

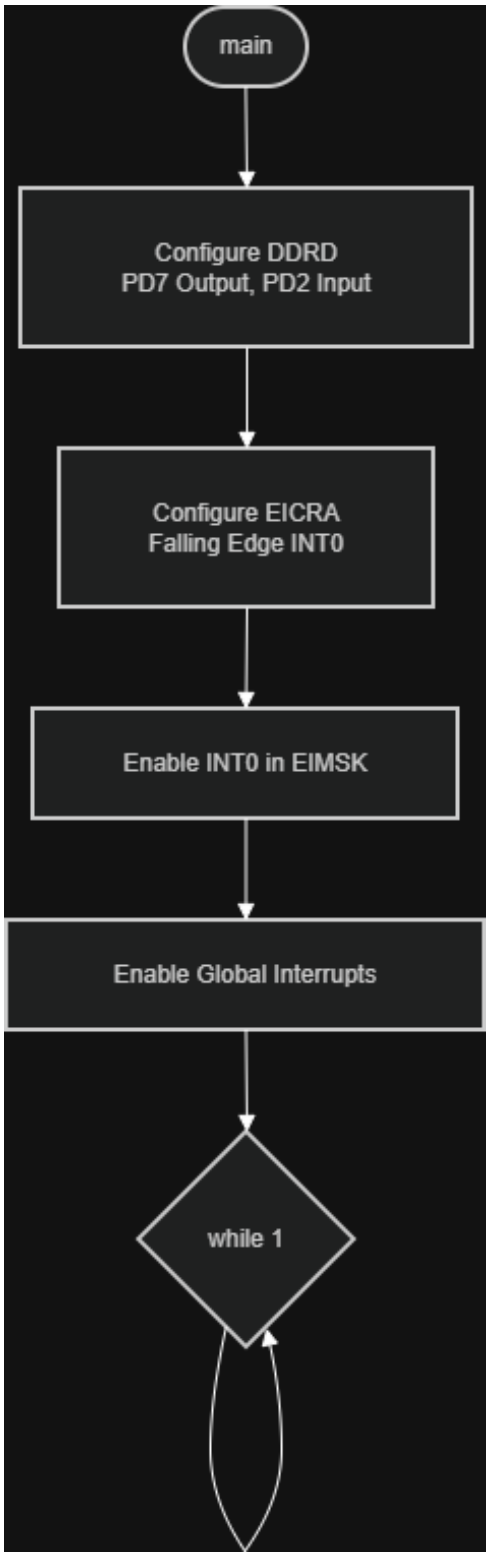


Figure 6: Flowchart for task 4

5.5 Circuit Design And Code Explanation

From the Figure 5 and 6, The circuit uses pin `PD7` to drive an LED through a current–limiting resistor, while pin `PD2` is connected to a push button that generates the external interrupt `INT0`. In the program, `PD7` is configured as an output and `PD2` as an input, after which the interrupt is set to trigger on a falling edge using the bits `ISC01` and `ISC00` in `EICRA`. When the button is pressed, the ISR `INT0_vect` toggles the LED on `PD7` and applies a 1 s delay, while the main loop remains empty because all functionality is handled inside the interrupt routine.

5.6 Code Reference

See Appendix D.

6 Task 5 – Traffic Light Using Software Delays

6.1 Task Summary

Task 5 implements a complete traffic-light sequence using software-based delays. Three LEDs are connected to PD2 (Red), PD3 (Yellow), and PD4 (Green). The sequence follows real-world timing:

1. Red ON for 4 s
2. Red + Yellow ON for 2 s
3. Green ON for 6 s
4. Yellow ON for 2 s

All delays are implemented using repeated `_delay_ms(10)` calls.

6.2 Handling Long Delays

The built-in `_delay_ms()` function cannot reliably handle long delays in a single call. To achieve multi-second delays, the program repeats 10 ms delays:

$$\text{long delay} = \text{repeat } _delay_ms(10)$$

Examples:

- $4\,\mathrm{s} = 400 \times 10\,\mathrm{ms}$
- $6\,\mathrm{s} = 600 \times 10\,\mathrm{ms}$

This approach:

- avoids timer configuration,
- ensures predictable timing,
- keeps the implementation simple.

6.3 Explanation of Operation

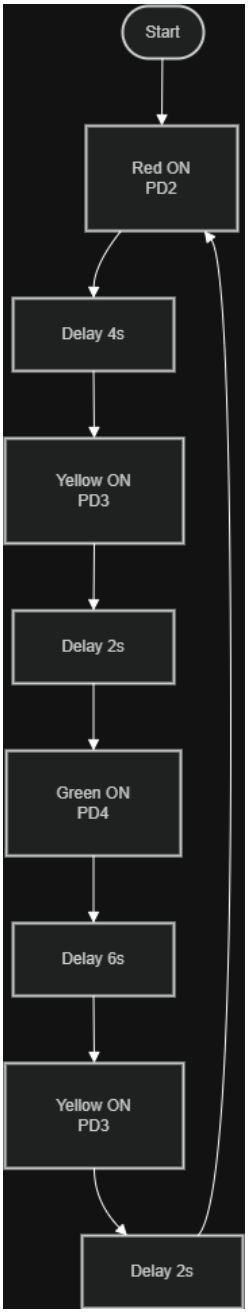


Figure 7: Flowchart for the traffic light sequence in Task 5.

Figure 7 shows, how the program controls a simple traffic–light system using pins `PD2`, `PD3`, and `PD4` as outputs. The sequence begins with the Red LED on `PD2` for 4 s, followed by the Yellow LED on `PD3` for 2 s, and then the Green LED on `PD4` for 6 s. After Green, the Yellow LED is activated again for 2 s before the cycle returns to Red. Each state uses a dedicated blocking delay function (such as `delay_4s()` or `delay_6s()`), meaning the CPU remains occupied during each delay, which is acceptable for this simple sequential task.

6.4 Traffic Light Circuit and Code

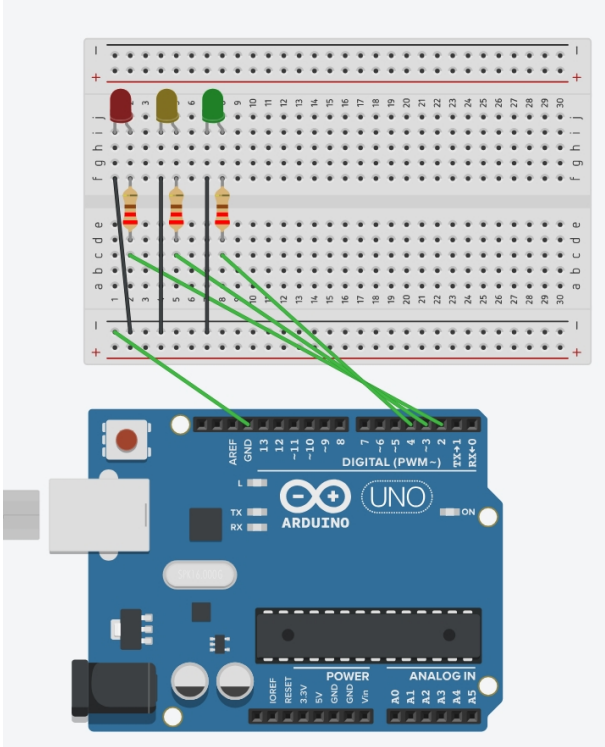


Figure 8: Flowchart for the traffic light sequence in Task 5 and Task 6.

From Figure 8, The circuit uses three LEDs to represent the Red, Yellow, and Green traffic lights. Each LED is connected in series with a current–limiting resistor, and the resistors are wired to the microcontroller pins `PD2`, `PD3`, and `PD4`. All LED cathodes are tied to the ground rail, which is connected to the microcontroller's GND pin. In the program, these pins are configured as outputs, and the software activates them in the order Red → Yellow → Green → Yellow to simulate a real traffic–light cycle. Each state uses a blocking delay function (such as `delay_4s()` or `delay_6s()`), meaning the CPU remains occupied during each timing interval. This simple sequential structure ensures that the LEDs turn on and off exactly according to the timing defined in the flowchart.

6.5 Code Reference

See Appendix E.

7 Task 6 – Cycling LEDs Using Button + Timer-Based Debouncing

7.1 Task Summary

Task 6 cycles through three LEDs (Red, Yellow, Green) each time a button is pressed. The LEDs are connected to PD2–PD4, and the button is connected to PD7 with an internal pull-up resistor. Timer1 generates a 10 ms periodic interrupt used for debouncing.

7.2 Design and Optimization

To ensure reliable button detection:

- Timer1 generates a 10 ms tick.
- A global tick counter increments inside the ISR.
- A 100 ms debounce delay is implemented using the tick counter.
- Falling-edge detection ensures one toggle per press.

7.3 Explanation of Operation

Task 6 Reference Image From the referenced link, The circuit uses three LEDs connected to pins `PD2`, `PD3`, and `PD4` through current–limiting resistors, with all cathodes tied to ground. These pins are configured as outputs in the program, and the software activates them in the sequence Red → Yellow → Green → Yellow to simulate a traffic light. Each state uses a blocking delay function, ensuring the CPU remains occupied during each timing interval. The main loop performs:

1. Read PD7.
2. Detect falling edge (HIGH → LOW).
3. Wait 100 ms using the tick counter.
4. Confirm button still pressed.
5. Increment LED index ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$).
6. Update LEDs accordingly.

This produces a responsive and stable LED-cycling system.

7.4 Code Reference

See Appendix F.

8 Task 7 – Interrupt-Based Communication Between Two Arduinos

8.1 Introduction

This task demonstrates interrupt-based communication between two Arduino boards using a shared LED as the signaling interface. Arduino 1 receives serial commands from the PC and controls the shared LED, while Arduino 2 monitors that LED through an interrupt and drives its own LED accordingly. The design ensures that only one Arduino drives the shared line, preventing electrical conflicts. With both main loops kept empty, all system behavior is handled entirely through interrupts, highlighting an event-driven architecture.

Explanation of Flowchart and Code (Task 7)

Task 7 Flowchart Image, The flowchart illustrates an interrupt-driven program where the microcontroller first configures the LED pin (PB5), initializes the USART module at 9600 baud, and enables global interrupts before entering an empty main loop. In the first code example, the `USART_RX` interrupt reads each received character from `UDR0`; receiving '1' turns the LED on, while '0' turns it off, meaning all LED control occurs inside the ISR. The main loop remains empty because the program is fully event-driven. The second code block demonstrates an external interrupt setup where PD2 is input and PD7 is output; each `INT0` trigger reads PD2 and updates PD7 accordingly. Both programs follow the same initialization sequence shown in the flowchart: configure hardware, enable interrupts, and let ISRs handle all real-time actions.

- Shared LED ON → Arduino 2 LED OFF
- Shared LED OFF → Arduino 2 LED ON

Both Arduinos use empty `while(1)` loops; all logic is interrupt-driven.

8.2 System Architecture

Figure 9 shows, The circuit consists of two Arduino Uno boards connected to a shared LED on a breadboard. Both Arduinos share a common ground, and the LED is wired in

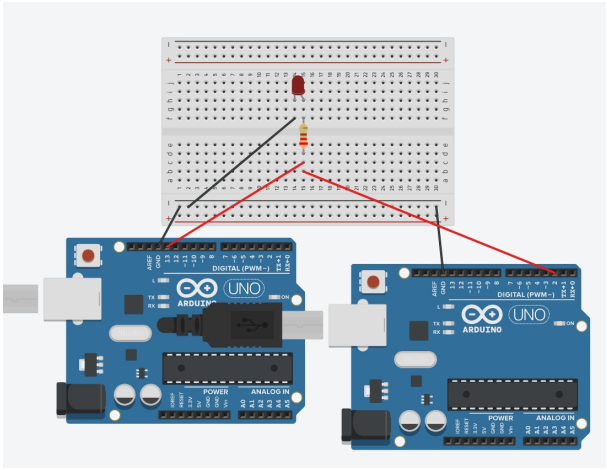


Figure 9: Two Arduino

series with a current-limiting resistor. The anode of the LED is connected to Arduino 1 through digital pin 13, allowing it to drive the LED based on received USART commands. The same LED node is also connected to Arduino 2 via digital pin 7, enabling it to sense the LED state as an input. The cathode of the LED returns to ground, completing the circuit. This shared-line configuration allows Arduino 1 to control the LED while Arduino 2 monitors the same signal for interrupt-driven processing.

8.3 Safety Considerations

- Only Arduino 1 drives the shared LED pin.
- Arduino 2 configures its sensing pin as input-only.
- A series resistor protects the shared LED.
- 10 k resistor prevents leakage between boards.

8.4 Code Reference

See Appendix G.

Appendix

A Task 1 Code

```
/*
 * LAB_4_task_1.c
 * Blinking LED using Timer1 Compare Match A interrupt
```

```
 */

#include <avr/io.h>
#include <avr/interrupt.h>

ISR(TIMER1_COMPA_vect)
{
    PORTB ^= (1 << PB5);
}

void timer1_init(void)
{
    TCCR1A = 0;
    TCCR1B = (1 << WGM12);
    OCR1A = 15624;
    TCCR1B |= (1 << CS12) | (1 << CS10);
    TIMSK1 |= (1 << OCIE1A);
}

int main(void)
{
    DDRB |= (1 << PB5);
    timer1_init();
    sei();

    while (1) { }
}
```

B Task 2 Code

```
/*
 * LAB_4_Task_2.c
 * UART-controlled blinking using Timer1 interrupt
 */

#define F_CPU 16000000UL
#define BAUD 9600
#define MYUBRR (F_CPU/16/BAUD - 1)

#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t blink_enable = 0;

ISR(USART_RX_vect)
{
```

```
    uint8_t data = UDR0;

    if (data == '1')
        blink_enable = 1;
    else if (data == '0')
    {
        blink_enable = 0;
        PORTB &= ~(1 << PB5);
    }
}

ISR(TIMER1_COMPA_vect)
{
    if (blink_enable)
        PORTB ^= (1 << PB5);
}

void timer1_init(void)
{
    TCCR1A = 0;
    TCCR1B = (1 << WGM12);
    OCR1A = 15624;
    TCCR1B |= (1 << CS12) | (1 << CS10);
    TIMSK1 |= (1 << OCIE1A);
}

void USART_Init(unsigned int ubrr)
{
    UBRR0H = (ubrr >> 8);
    UBRR0L = ubrr;
    UCSR0B = (1 << RXEN0) | (1 << RXCIE0);
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

int main(void)
{
    DDRB |= (1 << PB5);
    USART_Init(MYUBRR);
    timer1_init();
    sei();

    while (1) { }
}
```

C Task 3 Code

```
/*
 * LAB_4_Task_3.c
 * Button-controlled toggle using polling + 10 ms Timer1 delay
 */

#define F_CPU 16000000UL
#include <avr/io.h>

void timer1_delay_10ms(void)
{
    TCNT1 = 0;
    TCCR1A = 0;
    TCCR1B = (1 << CS12) | (1 << CS10);

    while (TCNT1 < 156) { }

    TCCR1B = 0;
}

int main(void)
{
    DDRB |= (1 << PB5);
    DDRD &= ~(1 << PD7);
    PORTD |= (1 << PD7);

    while (1)
    {
        if (!(PIND & (1 << PD7)))
        {
            timer1_delay_10ms();

            if (!(PIND & (1 << PD7)))
            {
                PORTB ^= (1 << PB5);

                while (!(PIND & (1 << PD7))) { }

                timer1_delay_10ms();
            }
        }
    }
}
```

D Task 4 Code

```
/*
 * LAB_4_Task_4.c
 * Button-controlled toggle using external interrupt INT0
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

ISR(INT0_vect)
{
    PORTD ^= (1 << PD7);
    _delay_ms(1000);
}

int main(void)
{
    DDRD |= (1 << PD7);
    DDRD &= ~(1 << PD2);

    EICRA |= (1 << ISC01);
    EICRA &= ~(1 << ISC00);

    EIMSK |= (1 << INT0);

    sei();

    while (1) { }
}
```

E Task 5 Code

```
/*
 * LAB_4_task_5.c
 * Traffic light using software delays
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

void delay_2s(void) { for (int i=0;i<200;i++) _delay_ms(10); }
```

```
void delay_4s(void) { for (int i=0;i<400;i++) _delay_ms(10); }
void delay_6s(void) { for (int i=0;i<600;i++) _delay_ms(10); }

int main(void)
{
    DDRD |= (1 << PD2) | (1 << PD3) | (1 << PD4);

    while (1)
    {
        PORTD |= (1 << PD2);
        PORTD &= ~((1 << PD3) | (1 << PD4));
        delay_4s();

        PORTD |= (1 << PD3);
        delay_2s();

        PORTD &= ~(1 << PD2);
        PORTD &= ~(1 << PD3);
        PORTD |= (1 << PD4);
        delay_6s();

        PORTD &= ~(1 << PD4);
        PORTD |= (1 << PD3);
        delay_2s();
    }
}
```

F Task 6 Code

```
/*
 * LAB_4_task_6.c
 * Cycling LEDs using button + timer-based debouncing
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t tick_10ms = 0;

ISR(TIMER1_COMPA_vect)
{
    tick_10ms++;
}

void timer1_init(void)
```

```
{
    TCCR1A = 0;
    TCCR1B = (1 << WGM12);
    OCR1A = 156;
    TCCR1B |= (1 << CS12) | (1 << CS10);
    TIMSK1 |= (1 << OCIE1A);
    sei();
}

void debounce_wait_100ms(void)
{
    tick_10ms = 0;
    while (tick_10ms < 10) { }
}

int main(void)
{
    DDRD |= (1 << PD2) | (1 << PD3) | (1 << PD4);
    DDRD &= ~(1 << PD7);
    PORTD |= (1 << PD7);

    timer1_init();

    uint8_t last_state = 1;
    uint8_t led_index = 0;

    PORTD |= (1 << PD2);

    while (1)
    {
        uint8_t current_state = (PIND & (1 << PD7)) ? 1 : 0;

        if (!current_state && last_state)
        {
            debounce_wait_100ms();

            if (!(PIND & (1 << PD7)))
            {
                led_index++;
                if (led_index > 2) led_index = 0;

                PORTD &= ~((1 << PD2) | (1 << PD3) | (1 << PD4));

                if (led_index == 0) PORTD |= (1 << PD2);
                if (led_index == 1) PORTD |= (1 << PD3);
                if (led_index == 2) PORTD |= (1 << PD4);
            }
        }
```

```
        last_state = current_state;
    }
}
```

G Task 7 Code

```
/*
 * LAB_4_task_7.c
 * Arduino 1: UART-controlled shared LED
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(USART_RX_vect)
{
    uint8_t c = UDR0;

    if (c == '1')
        PORTB |= (1 << PB5);
    else if (c == '0')
        PORTB &= ~(1 << PB5);
}

void USART_init(void)
{
    UBRR0H = 0;
    UBRR0L = 103;
    UCSR0B = (1 << RXEN0) | (1 << RXCIE0);
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

int main(void)
{
    DDRB |= (1 << PB5);
    USART_init();
    sei();

    while (1) { }
}


/*
 * Conceptual Arduino 2 Code (for report only)
```

```
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect)
{
    uint8_t shared = (PIND & (1 << PD2));

    if (shared)
        PORTD &= ~(1 << PD7);
    else
        PORTD |= (1 << PD7);
}

int main(void)
{
    DDRD |= (1 << PD7);
    DDRD &= ~(1 << PD2);

    EICRA |= (1 << ISC00);
    EIMSK |= (1 << INT0);

    sei();

    while (1) { }
}
```